

Les déclencheurs dans MYSQL

Les déclencheurs dans MYSQL

Définition :

Un déclencheur (Trigger) est une procédure stockée, dans la base de données, qui s'appelle automatiquement à chaque fois qu'un événement spécial se produit sur la table à laquelle le déclencheur est attaché.

On peut utiliser un déclencheur pour valider les données, enregistrer les anciennes et les nouvelles valeurs dans une table d'audit (log) ou s'assurer que les règles métier sont respectées. Par exemple, un déclencheur peut être invoqué lorsqu'une ligne est insérée dans une table spécifique ou lorsque certaines colonnes de la table sont mises à jour.

Syntaxe :

On utilise l'instruction **CREATE TRIGGER** pour créer un déclencheur dans MYSQL à l'aide de la syntaxe suivante :

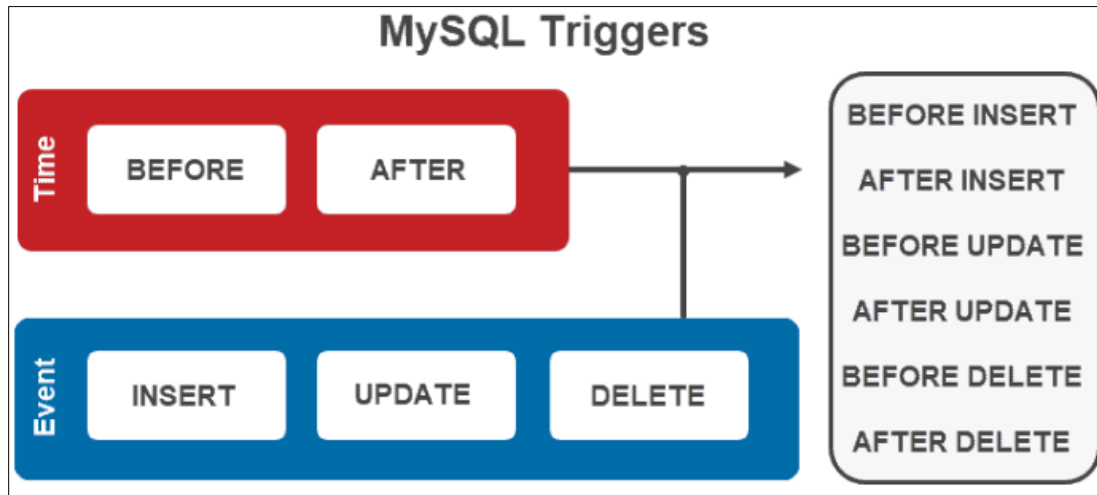
```
DELIMITER //
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name FOR EACH ROW
BEGIN
    trigger_code;
END//
DELIMITER ;
```

- **{BEFORE | AFTER}** : Ceci spécifie quand le déclencheur sera exécuté (avant ou après).
- **{INSERT | UPDATE | DELETE}** : ceci spécifie l'opération (Ajout, modification ou suppression).
- **ON table_name** : Ceci spécifie le nom de la table associée au déclencheur.
- **FOR EACH ROW** : Ceci spécifie que le déclencheur sera exécuté pour chaque ligne affectée.
- **trigger_code** : Ceci fournit les opérations à effectuer lorsque le déclencheur est déclenché.

Utilisation des déclencheurs :

Chaque déclencheur associé à une table selon deux facteurs :

1. Temps : **BEFORE** ou **AFTER**.
2. Événement : **INSERT**, **UPDATE** ou **DELETE**.



Les mots clés OLD et NEW :

Lors de la manipulation des déclencheurs, vous êtes amené à utiliser deux mots clé : **OLD** et **NEW** :

- **OLD.column_name** : utilisée pour accéder aux données de ligne qui sont remplacées par la mise à jour.
- **NEW.column_name** : permet d'accéder aux données de ligne entrantes qui remplaceront l'ancienne ligne.

Selon le type de déclencheur créé, les lignes **OLD** et **NEW** peuvent ne pas être disponibles :

- Pour un **INSERT TRIGGER** : Accès possible a **NEW** uniquement.
- Pour un **UPDATE TRIGGER** : Accès aux deux pseudo lignes **OLD** et **NEW**.
- Pour un **DELETE TRIGGER** : Accès possible a **OLD** uniquement.

Exemple 1 : BEFORE INSERT

On commence par la création de la table **employee** suivante :

```
CREATE TABLE employee(  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(45) NOT NULL,  
    occupation VARCHAR(35) NOT NULL,  
    working_date DATE,  
    working_hours VARCHAR(10)  
);  
INSERT INTO employee(name,occupation,working_date,working_hours) VALUES  
( 'RAJI', 'Scientist', '2021-10-04', 12),  
( 'ALAMI', 'Engineer', '2020-10-04', 10),  
( 'BEN ALI', 'Actor', '2020-10-04', 13),  
( 'NAJI', 'Doctor', '2022-10-04', 14),  
( 'BADDI', 'Teacher', '2020-10-04', 12),  
( 'IBRAHIMI', 'Business', '2020-10-04', 11);  
SELECT * FROM employee;
```

	id	name	occupation	working_date	working_hours
▶	1	RAJI	Scientist	2021-10-04	12
	2	ALAMI	Engineer	2020-10-04	10
	3	BEN ALI	Actor	2020-10-04	13
	4	NAJI	Doctor	2022-10-04	14
	5	BADDI	Teacher	2020-10-04	12
	6	IBRAHIMI	Business	2020-10-04	11
*	NULL	NULL	NULL	NULL	NULL

Nous allons maintenant créer un trigger **BEFORE INSERT**. Ce déclencheur est appelé automatiquement et va mettre le champ **working_hours= 0** si quelqu'un essaie d'insérer **working_hours < 0** :

```

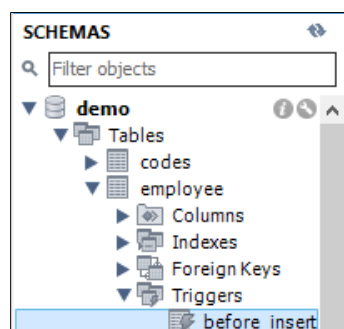
DELIMITER //
CREATE TRIGGER before_insert
BEFORE INSERT
ON employee
FOR EACH ROW
BEGIN
    IF NEW.working_hours < 0 THEN
        SET NEW.working_hours = 0;
    END IF;
END //
DELIMITER ;

INSERT INTO employee(name,occupation,working_date,working_hours) VALUES
('BEN ALI', 'Doctor', '2020-10-04', 10), ('NOURI', 'Teacher', '2020-10-04', -12);
SELECT * FROM employee;

```

	id	name	occupation	working_date	working_hours
▶	1	RAJI	Scientist	2021-10-04	12
	2	ALAMI	Engineer	2020-10-04	10
	3	BEN ALI	Actor	2020-10-04	13
	4	NAJI	Doctor	2022-10-04	14
	5	BADDI	Teacher	2020-10-04	12
	6	IBRAHIMI	Business	2020-10-04	11
	7	BEN ALI	Doctor	2020-10-04	10
	8	NOURI	Teacher	2020-10-04	0
*	NULL	NULL	NULL	NULL	NULL

Pour afficher le déclencheur **before_insert** créée :



1. Sélectionner la base de données, ici il s'agit de la base de données **demo**.
2. Double-cliquer dessus et afficher le sous-menu contenant les tables
3. Cliquez sur le sous-menu Tables et sélectionnez la table sur laquelle vous avez créé un déclencheur. Ici dans notre cas la table **employee**.
4. En cliquant sur le sous-menu Triggers, nous pouvons noter le déclencheur **before_insert**.

Exemple 2 : AFTER INSERT

Maintenant on va vider la table **employee** et créer une autre table **employee_detail** :

```
TRUNCATE employee;
CREATE TABLE employee_detail(
    emp_id PRIMARY KEY,
    emp_name VARCHAR(45) NOT NULL,
    emp_occupation VARCHAR(35) NOT NULL,
    emp_working_date DATE,
    emp_working_hours VARCHAR(10),
    Lasinserted TIME
);
```

Nous allons maintenant créer un trigger **AFTER INSERT**. Ce déclencheur est appelé automatiquement et va permettre d'enregistrer les détails de chaque employé dans la table **employee_detail** à chaque insertion dans la table **employee** :

```
DELIMITER //
CREATE TRIGGER after_insert
AFTER INSERT
ON employee
FOR EACH ROW
BEGIN
    INSERT INTO employee_detail VALUES (NEW.id, NEW.name, NEW.occupation,
        NEW.working_date, NEW.working_hours, CURRENT_TIME());
END //
DELIMITER ;

INSERT INTO employee (name, occupation, working_date, working_hours) VALUES
('RAJI', 'Scientist', '2021-10-04', 12);
INSERT INTO employee (name, occupation, working_date, working_hours) VALUES
('NAJI', 'Doctor', '2022-10-04', 14);
INSERT INTO employee (name, occupation, working_date, working_hours) VALUES
('IBRAHIMI', 'Business', '2020-10-04', 11);

SELECT * FROM employee_detail;
```

	emp_id	emp_name	emp_occupation	emp_working_date	emp_working_hours	Lasinserted
▶	1	RAJI	Scientist	2021-10-04	12	14:27:10
	2	NAJI	Doctor	2022-10-04	14	14:29:48
	3	IBRAHIMI	Business	2020-10-04	11	14:30:06
*	NULL	NULL	NULL	NULL	NULL	NULL

Exemple 3 : BEFORE UPDATE

On va créer un déclencheur qui n'autorise pas la modification du champ **working_hours** par une valeur inférieure ou égale à la valeur actuelle dans la table **employee** :

```
DELIMITER //
CREATE TRIGGER before_update
BEFORE UPDATE
ON employee
FOR EACH ROW

BEGIN
    IF NEW.working_hours <= OLD.working_hours THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT="Le nombre d'heures de
travail ne doit pas être inférieur ou égale à la valeur actuelle";
    END IF;
END //
DELIMITER ;
UPDATE employee SET working_hours=13 WHERE id=1;
UPDATE employee SET working_hours=12 WHERE id=1;
```

❌ 31 14:48:14 UPDATE employee SET working_hours=12 WHERE id=1

Error Code: 1644. Le nombre d'heures de travail ne doit pas être inférieur ...

Exemple 4 : AFTER UPDATE

On va créer les deux tables **students** et **students_log** :

```
CREATE TABLE students(
    id INT PRIMARY KEY,
    name VARCHAR(45) NOT NULL,
    class INT NOT NULL,
    email VARCHAR(65) NOT NULL
);

INSERT INTO students VALUES
(1,'RAJI', 6, 'raji@abc.com'),
(2,'ALAMI', 7, 'alami@abc.com'),
(3,'BEN ALI', 4, 'benali@abc.com'),
(4,'NAJI', 5, 'naji@abc.com');

CREATE TABLE students_log(
    mysql_user VARCHAR(45) NOT NULL,
    student_id INT NOT NULL,
    description VARCHAR(150) NOT NULL
);
```

Nous allons créer un déclencheur **AFTER UPDATE** qui se déclenche lorsque seulement une mise à jour du champ **class** vers le niveau supérieur de la table **students** est effectuée et dans ce cas les informations de la mise à jour sont enregistrées dans la table **students_log** :

```
DELIMITER //
CREATE TRIGGER after_update
AFTER UPDATE
ON students
FOR EACH ROW
BEGIN
    IF NEW.class != OLD.class AND NEW.class= OLD.class + 1 THEN
        INSERT INTO students_log VALUES(USER(), OLD.id, CONCAT('Previous
Class :', OLD.class, ' Present Class : ', NEW.class));
    ELSE
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='ERROR class';
    END IF;
END //
DELIMITER ;

UPDATE students SET class=5 WHERE id=1;
```

✖ 67 16:07:08 UPDATE students SET class=5 WHERE id=1 Error Code: 1644. ERROR class

```
UPDATE students SET class=8 WHERE id=2;
SELECT * FROM students_log;
```

	mysql_user	student_id	description
▶	root@localhost	2	Previous Class : 7 Present Class : 8

Exemple 5 : BEFORE DELETE

On va créer les deux tables **salaries** et **salary_archives** :

```
CREATE TABLE salaries (
    emp_num INT PRIMARY KEY,
    valid_from DATE NOT NULL,
    amount DOUBLE NOT NULL DEFAULT 0
);

INSERT INTO salaries (emp_num, valid_from, amount) VALUES
(102, '2020-01-10', 45000), (103, '2020-01-10', 65000),
(105, '2020-01-10', 55000),
(107, '2020-01-10', 70000),
(109, '2020-01-10', 40000);

CREATE TABLE salary_archives (
    id INT PRIMARY KEY AUTO_INCREMENT,
    emp_num INT,
    valid_from DATE NOT NULL,
    amount DOUBLE NOT NULL DEFAULT 0,
    deleted_time TIMESTAMP DEFAULT NOW()
);

SELECT * FROM salaries;
```

	emp_num	valid_from	amount
▶	102	2020-01-10	45000
	103	2020-01-10	65000
	105	2020-01-10	55000
	107	2020-01-10	70000
	109	2020-01-10	40000
*	NULL	NULL	NULL

Nous allons créer un déclencheur **BEFORE DELETE** qui insère les informations d'un salarié dans la table **salary_archives** avant chaque suppression d'une ligne de la table **salaries** :

```
DELIMITER //
CREATE TRIGGER before_delete
BEFORE DELETE
ON salaries
FOR EACH ROW
BEGIN
    INSERT INTO salary_archives(emp_num, valid_from, amount)
    VALUES (OLD.emp_num, OLD.valid_from, OLD.amount);
END //
DELIMITER ;

DELETE FROM salaries WHERE emp_num=105;
SELECT * FROM salary_archives;
```

	id	emp_num	valid_from	amount	deleted_time
▶	1	105	2020-01-10	55000	2023-02-07 18:00:47
*	NULL	NULL	NULL	NULL	NULL

Exemple 6 : AFTER DELETE

En plus de la table **salaries**, on va créer une autre table **total_salary_budget** :

```
CREATE TABLE total_salary_budget(
    total_budget DOUBLE NOT NULL
);
SELECT * FROM salaries;
```

	emp_num	valid_from	amount
▶	102	2020-01-10	45000
	103	2020-01-10	65000
	107	2020-01-10	70000
	109	2020-01-10	40000
*	NULL	NULL	NULL

```
INSERT INTO total_salary_budget (total_budget) SELECT SUM(amount) FROM salaries;
SELECT * FROM total_salary_budget;
```

	total_budget
▶	220000

Nous allons créer un déclencheur **AFTER DELETE** qui va mettre à jour le champ **total_budget** de la table **total_salary_budget** après chaque suppression d'une ligne de la table **salaries** :

```

DELIMITER //
CREATE TRIGGER after_delete
AFTER DELETE
ON salaries
FOR EACH ROW
BEGIN
    UPDATE total_salary_budget SET total_budget = total_budget - OLD.amount;
END //
DELIMITER ;

DELETE FROM salaries WHERE emp_num BETWEEN 102 AND 109;
SELECT * FROM total_salary_budget;

```

	total_budget
▶	0

Autres informations sur les déclencheurs

Pour lister les déclencheurs de la base de données, on utilise :

```
SHOW TRIGGERS;
```

Trigger	Event	Table	Statement	Timing	Created	sql_mode	Definer	character_set_client	collation_connection	Database Collation
▶ before_insert	INSERT	employee	BEGIN IF NEW.working_hours < 0 THEN SET NE...	BEFORE	2023-02-07 16:39:15.63	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
after_insert	INSERT	employee	BEGIN INSERT INTO employee_detail VALUES(N...	AFTER	2023-02-07 16:46:49.44	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
trigger1	UPDATE	employee	begin if new.working_hours <= old.working_ho...	BEFORE	2023-02-07 16:54:07.83	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
before_delete	DELETE	salaries	BEGIN INSERT INTO salary_archives(emp_num,...	BEFORE	2023-02-07 17:58:20.72	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
after_delete	DELETE	salaries	BEGIN UPDATE total_salary_budget SET total_b...	AFTER	2023-02-07 18:18:09.97	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...
trigger2	UPDATE	students	begin if new.class = old.class + 1 then insert...	AFTER	2023-02-07 17:06:52.05	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900...

Pour supprimer un trigger, on utilise :

```
DROP TRIGGER [IF EXISTS] database_name.trigger_name;
```

Remarque : Si on supprime une table, MYSQL supprimera automatiquement tous les déclencheurs associés à la table.

Il n'existe pas une instruction pour modifier un trigger, on doit le supprimer et le recréer. Penser à utiliser l'instruction **LOCK TABLE** et **UNLOCK TABLES** :

```

LOCK TABLE t1 WRITE ;
DROP TRIGGER t1_bi ;
DELIMITER $$
CREATE TRIGGER t1_bi BEFORE INSERT ON t1 FOR EACH ROW
BEGIN
    ...
END $$
DELIMITER ;
UNLOCK TABLES ;

```